

An Introduction to Variational Autoencoders

El siguiente contenido está basado en el documento “*An Introduction to Variational Autoencoders*” de Diederik P. Kingma y Max Welling (2019).

Introducción

Los Variational Autoencoders (VAE) entregan un framework para aprender modelos profundos de variables latentes y los modelos de inferencia correspondientes.

Suponga que queremos modelar la distribución conjunta de unas variables observadas. Sea $\mathbf{x}$ un vector de estas variables. De hecho, asumiremos que $\mathbf{x}$ es una muestra de un *proceso desconocido* y *subyacente*, con distribución real $p^*(\mathbf{x})$ desconocida. Así, lo que buscamos es aproximar esta p^* .

Para aproximar p^* , partimos de un modelo p_{θ} , cuyos parámetros en θ ajustaremos de modo que

$$p_{\theta}(\mathbf{x}) \approx p^*(\mathbf{x}).$$

De esta manera, tendremos un modelo generativo tal que

$$\mathbf{x} \sim p_{\theta}(\mathbf{x}).$$

Modelos Condicionales

A veces tendremos conocimiento sobre la distribución de los datos, lo que podremos incluir con una distribución *a priori*. Los modelos que involucren este tipo de conocimiento se conocen como **Modelos Condicionales**. Asuma que queremos aproximar una distribución sobre los valores de una variable $\mathbf{y}$, dados unos datos $\mathbf{x}$ (observados, el **input**). Es decir, queremos aproximar $p^*(\mathbf{y}|\mathbf{x})$ ajustando $p_{\theta}(\mathbf{y}|\mathbf{x})$.

Ejemplo 01. Un ejemplo clásico es la clasificación de imágenes. Una imagen se toma como input, $\mathbf{x}$, y su clase se piensa como valor de $\mathbf{y}$. ■

Redes Neuronales

Asumiremos que las **redes neuronales** son redes feed-forward diferenciables. Dos características de ellas son que:

- (1) Permiten *aproximar funciones*, en particular, densidades.
- (2) Son *escalables* a grandes modelos y grandes datasets, pues admiten optimización con SGD.

Cuando estas redes neuronales tienen muchas capas ocultas, decimos que estamos ante un problema de **deep learning**. Denotaremos estas redes como la función vectorial (e.g. un vector de probabilidades de cada clase)

$$\text{NeuralNet}(\cdot).$$

Ejemplo 02. La clasificación de imágenes con redes neuronales (LeCun *et al.*, 2016) se ha realizado parametrizando una distribución categórica $p_{\theta}(\mathbf{y}|\mathbf{x})$ sobre una clase de etiquetas $\mathbf{y}$, condicionado a una imagen $\mathbf{x}$. (Los pesos de la red neuronal corresponden a θ .) Se define

$$\mathbf{p} = \text{NeuralNet}(\mathbf{x}),$$

esto es, la salida que produce la red neuronal que, en este caso, corresponde a un vector de probabilidades de clases (ergo $\sum_i p_i = 1$ y $p_i \geq 0$). Luego, se piensa

$$p_{\theta}(\mathbf{y}|\mathbf{x}) = \text{Categorical}(\mathbf{y}; \mathbf{p}),$$

donde Categorical es una distribución categórica parametrizada por $\mathbf{p}$. ■

Modelos Gráficos Dirigidos

Los **modelos probabilísticos dirigidos**, **modelos gráficos probabilísticos** o **redes bayesianas** son modelos donde todas las variables están organizadas en un digrafo acíclico:

- Los nodos representan a las variables.
- Las aristas representan relaciones de independencia.

Denotaremos con $\text{Pa}(x_j)$ los padres de la variable x_j . Aquí, la distribución sobre las variables resulta en

$$p_{\theta}(x_1, \dots, x_M) = \prod_{j=1}^M p_{\theta}(x_j | \text{Pa}(x_j)).$$

Nota 03. Si x_j no tiene padres, i.e. es una raíz, $\text{Pa}(x_j) = \emptyset$, pero la distribución se piensa incondicional simplemente. ■

Dataset y Verosimilitud

Nuestro **dataset** es

$$\mathcal{D} = \{x^{(i)}\}_{i=1}^N =: x^{(1:N)}.$$

Dado un dataset $\mathcal{D}$, denotamos con $N_{\mathcal{D}}$ su tamaño. Muchas veces asumimos las observaciones $x^{(i)}$ i.i.d. Así, podemos calcular la probabilidad (densidad) de los datos según el modelo. En particular, podemos obtener la log-probabilidad asociada

$$\log p_{\theta}(\mathcal{D}) = \sum_{i=1}^N \log p_{\theta}(x^{(i)}).$$

El criterio más común para los modelos es el de la **Máxima Verosimilitud (ML)**, esto es, encontrar el θ que maximiza la verosimilitud (o la log-verosimilitud). (Que es equivalente a la minimización de la divergencia de Kullback-Leibler). Para ello, se usa **Stochastic Gradient Descent (SGD)**¹ que usa minibatches $\mathcal{M} \subset \mathcal{D}$ y define el estimador

$$\frac{1}{N_{\mathcal{M}}} \log p_{\theta}(\mathcal{M}) \simeq \frac{1}{N_{\mathcal{D}}} \log p_{\theta}(\mathcal{D})$$

(“ $\simeq$ ” indica que un miembro de la ecuación es un estimador insesgado del otro). Este estimador es diferenciable, por lo que podemos estimar

$$\frac{1}{N_{\mathcal{M}}} \nabla_{\theta} \log p_{\theta}(\mathcal{M}) \simeq \frac{1}{N_{\mathcal{D}}} \nabla_{\theta} \log p_{\theta}(\mathcal{D}).$$

Variables Latentes y Modelos Profundos Latentes (DLVMs)

El método presentado en la sección anterior es válido cuando las variables del modelo dirigido son *totalmente observadas*, pero podemos extenderlo a modelos dirigidos con variables latentes. Las **variables latentes** (denotadas con z) son variables que son parte del modelo, pero que no observamos, por tanto, no forman parte del dataset.

En modelamiento incondicional, la distribución conjunta es $p_{\theta}(x, z)$. Para conocer $p_{\theta}(x)$,

$$p_{\theta}(x) = \int p_{\theta}(x, z) dz = \int p_{\theta}(x|z)p_{\theta}(z) dz.$$

(Recuerda que $p_{\theta}(x, z) = p_{\theta}(z|x)p_{\theta}(x) = p_{\theta}(x|z)p_{\theta}(z)$.)

¹Podríamos usar **Batch Gradient Descent** considerando al maximizar siempre todo $\mathcal{D}$, pero el costo sería muy alto para datasets grandes (lineal).

Cuando la distribución de un modelo de variable latente, $p_{\theta}(\mathbf{x}, \mathbf{z})$, está parametrizada por una red neuronal, usamos el término **Modelo Profundo de Variable Latente** (*deep latent variable model*, *DLVM*). Una ventaja de los DLVMs es su expresividad: la distribución marginal de $\mathbf{x}$ puede ser muy compleja, aun cuando la distribución condicional $p_{\theta}(\mathbf{x}|\mathbf{z})$ o la *a priori* $p_{\theta}(\mathbf{z})$ son simples.

Intratabilidades

La principal dificultad de los DLVMs es que la marginal $p_{\theta}(\mathbf{x}) = \int p_{\theta}(\mathbf{x}, \mathbf{z}) d\mathbf{z}$ es intratable, pues no tiene una fórmula analítica ni un estimador eficiente. Por esto, no podemos diferenciar respecto a θ para optimizarla (que es nuestro objetivo), esto es, no podemos aplicar SGD.

La intratabilidad de $p_{\theta}(\mathbf{x})$, recordando que

$$p_{\theta}(\mathbf{x}, \mathbf{z}) = p_{\theta}(\mathbf{z}|\mathbf{x})p_{\theta}(\mathbf{x}),$$

Provoca que $p(\mathbf{z}|\mathbf{x})$ sea intratable, en principio. Es decir, ambas densidades son intratables en DLVMs.

Para resolver esto, las *técnicas aproximadas de inferencia* nos permitirán aproximar $p_{\theta}(\mathbf{z}|\mathbf{x})$ y $p_{\theta}(\mathbf{x})$ en los DLVMs. Los métodos tradicionales son relativamente caros.

Variational Autoencoders

Superaremos las intratabilidades de los DLMVs con los VAEs.

Encoder

Definimos el **encoder**, **modelo de reconocimiento** o **modelo de inferencia** $q_{\phi}(\mathbf{z}|\mathbf{x})$. Aquí ϕ indica los parámetros del modelo, llamados **parámetros variacionales**. Buscaremos que esta distribución sea lo más parecida a la posteriori $p_{\theta}(\mathbf{z}|\mathbf{x})$, es decir,

$$q_{\phi}(\mathbf{z}|\mathbf{x}) \approx p_{\theta}(\mathbf{z}|\mathbf{x}).$$

Con ella podremos más adelante optimizar $p_{\theta}(\mathbf{x})$.

El modelo de inferencia puede ser prácticamente cualquier modelo gráfico dirigido,

$$q_{\phi}(\mathbf{z}|\mathbf{x}) = q_{\phi}(z_1, \dots, z_M|\mathbf{x}) = \prod_{j=1}^M q_{\phi}(z_j|\text{Pa}(z_j), \mathbf{x}).$$

Además, q_{ϕ} puede ser parametrizada por una red neuronal profunda.

Ejemplo 04. Podríamos tener

$$q_{\phi}(\mathbf{z}|\mathbf{x}) = \text{Normal}(\mathbf{z}; \boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma})),$$

donde

$$(\boldsymbol{\mu}, \log \boldsymbol{\sigma}) = \text{EncoderNeuralNet}_{\phi}(\mathbf{x}).$$

Usaremos un único encoder neuronal para realizar inferencia posterior sobre *todos* los puntos del dataset, a diferencia de lo que ocurre con otros métodos de inferencia variacional, donde los parámetros de ϕ se optimizan separadamente *para cada punto* del dataset. Nuestra técnica se conoce como **inferencia variacional amortizada**. Esto nos permitirá ser más eficientes y aprovechar el SGD.

Evidence Lower Bound (ELBO)

Para cada elección de encoder $q_{\phi}(\mathbf{z}|\mathbf{x})$ (con ϕ ya fijo),

$$\begin{aligned}
\log p_{\theta}(\mathbf{x}) &= \int q_{\phi}(\mathbf{z}|\mathbf{x}) \cdot \log p_{\theta}(\mathbf{x}) \, d\mathbf{z} \\
&= E_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x})] \\
&= E_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \left(\frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right) \right] \\
&= E_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \left(\frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \cdot \frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{p_{\theta}(\mathbf{z}|\mathbf{x})} \right) \right] \\
&= E_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \left(\frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right) \right] + E_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \left(\frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{p_{\theta}(\mathbf{z}|\mathbf{x})} \right) \right].
\end{aligned}$$

Definimos **Evidence Lower Bound (ELBO)**

$$\begin{aligned}
\mathcal{L}_{\theta, \phi}(\mathbf{x}) &:= E_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \left(\frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right) \right] \\
&= E_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}, \mathbf{z}) - \log q_{\phi}(\mathbf{z}|\mathbf{x})]
\end{aligned}$$

y la **divergencia de Kullback-Leibler (KL)** entre $q_{\phi}(\mathbf{z}|\mathbf{x})$ y $p_{\theta}(\mathbf{z}|\mathbf{x})$ como

$$\begin{aligned}
D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p_{\theta}(\mathbf{z}|\mathbf{x})) &:= E_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \left(\frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{p_{\theta}(\mathbf{z}|\mathbf{x})} \right) \right] \\
&= E_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\log q_{\phi}(\mathbf{z}|\mathbf{x}) - \log p_{\theta}(\mathbf{z}|\mathbf{x})].
\end{aligned}$$

De esta manera,

$$\log p_{\theta}(\mathbf{x}) = \mathcal{L}_{\theta, \phi}(\mathbf{x}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p_{\theta}(\mathbf{z}|\mathbf{x})).$$

Nótese que

$$D_{\text{KL}} \geq 0,$$

por lo que

$$\log p_{\theta}(\mathbf{x}) \geq \mathcal{L}_{\theta, \phi}(\mathbf{x}).$$

Además, observe que la divergencia KL determina dos distancias:

1. La distancia entre la posterior real, $p_{\theta}(\mathbf{z}|\mathbf{x})$, y su aproximación, $q_{\phi}(\mathbf{z}|\mathbf{x})$, por la definición que tiene.
2. El *gap* entre la verosimilitud marginal $\log p_{\theta}(\mathbf{x})$ y la ELBO $\mathcal{L}_{\theta, \phi}$. (Llamado **ajuste de la cota.**)

Es por esto que intentaremos maximizar $\mathcal{L}_{\theta, \phi}$ con respecto a sus parámetros θ, ϕ , de modo que ambas distancias se hagan lo más pequeñas posibles, de manera aproxima.

Stochastic Gradient-Based Optimization for ELBO

La ELBO permite SGD sobre sus parámetros.

Dado un dataset $\mathcal{D} = \mathbf{x}^{(1:N)}$, la ELBO objetivo es

$$\mathcal{L}_{\theta, \phi}(\mathcal{D}) := \sum_{i=1}^N \mathcal{L}_{\theta, \phi}(\mathbf{x}^{(i)}).$$

El gradiente individual $\nabla_{\theta, \phi} \mathcal{L}_{\theta, \phi}(\mathbf{x}^{(i)})$ es en general intratable, pero existen buenos estimados insesgados $\tilde{\nabla}_{\theta, \phi} \mathcal{L}_{\theta, \phi}(\mathbf{x}^{(i)})$.

Respecto a θ , es directo:

$$\begin{aligned}\nabla_{\theta} \mathcal{L}_{\theta, \phi}(\mathbf{x}) &= \nabla_{\theta} E_{z \sim q_{\phi}(z|\mathbf{x})} [\log p_{\theta}(z|\mathbf{x}) - \log q_{\phi}(z|\mathbf{x})] \\ &= \nabla_{\theta} \int q_{\phi}(z|\mathbf{x}) [\log p_{\theta}(z|\mathbf{x}) - \log q_{\phi}(z|\mathbf{x})] \mathrm{d}z \\ &\xrightarrow{\text{Leibniz}} \int \nabla_{\theta} \{q_{\phi}(z|\mathbf{x}) [\log p_{\theta}(z|\mathbf{x}) - \log q_{\phi}(z|\mathbf{x})]\} \mathrm{d}z \\ &= \int q_{\phi}(z|\mathbf{x}) \cdot \nabla_{\theta} [\log p_{\theta}(z|\mathbf{x}) - \log q_{\phi}(z|\mathbf{x})] \mathrm{d}z \\ &= E_{z \sim q_{\phi}(z|\mathbf{x})} [\nabla_{\theta} (\log p_{\theta}(z|\mathbf{x}) - \log q_{\phi}(z|\mathbf{x}))].\end{aligned}$$

Tomando un $z \sim q_{\phi}(z|\mathbf{x})$, se obtiene un estimador simple de Monte Carlo

$$\nabla_{\theta} \mathcal{L}_{\theta, \phi}(\mathbf{x}) \simeq \nabla_{\theta} (\log p_{\theta}(z|\mathbf{x}) - \log q_{\phi}(z|\mathbf{x})) = \nabla_{\theta} \log p_{\theta}(z|\mathbf{x}).$$

Respecto a ϕ es más difícil, pues q_{ϕ} depende justamente de ϕ . Es decir,

$$\begin{aligned}\nabla_{\phi} \mathcal{L}_{\theta, \phi}(\mathbf{x}) &= \nabla_{\phi} E_{z \sim q_{\phi}(z|\mathbf{x})} [\log p_{\theta}(z|\mathbf{x}) - \log q_{\phi}(z|\mathbf{x})] \\ &\neq E_{z \sim q_{\phi}(z|\mathbf{x})} [\nabla_{\phi} (\log p_{\theta}(z|\mathbf{x}) - \log q_{\phi}(z|\mathbf{x}))].\end{aligned}$$

Para esto, usaremos el *truco de reparametrización*.

Reparametrization Trick

Expresaremos $z \sim q_{\phi}(z|\mathbf{x})$ como una transformación g diferenciable e invertible de otra variable ϵ , independiente de $\mathbf{x}$ y de ϕ , y con densidad $p(\epsilon)$:

$$z = g(\epsilon, \phi, \mathbf{x}).$$

Con esto, para cualquier función f ,

$$E_{z \sim q_{\phi}(z|\mathbf{x})} [f(z)] = \int q_{\phi}(z|\mathbf{x}) f(z) \mathrm{d}z.$$

Pero,

$$\mathrm{d}z = \left| \det \left(\frac{\partial g}{\partial \epsilon} \right) \right| \mathrm{d}\epsilon$$

y

$$q(z|\mathbf{x}) = \left| \det \left(\frac{\partial g}{\partial \epsilon} \right) \right|^{-1} p(\epsilon),$$

así que

$$q(z|\mathbf{x}) \mathrm{d}z = p(\epsilon) \mathrm{d}\epsilon.$$

Por lo tanto,

$$\begin{aligned}
E_{z \sim q_\phi(z|x)}[f(z)] &= \int q_\phi(z|x) f(z) dz \\
&= \int f(g(\epsilon, \phi, x)) p(\epsilon) d\epsilon \\
&= E_{\epsilon \sim p(\epsilon)}[f(z)].
\end{aligned}$$

Ahora se puede hacer

$$\begin{aligned}
\nabla_\phi E_{z \sim q_\phi(z|x)}[f(z)] &= \nabla_\phi E_{\epsilon \sim p(\epsilon)}[f(z)] \\
&= \int \nabla_\phi \{f(g(\epsilon, \phi, x)) p(\epsilon)\} d\epsilon \\
&= \int p(\epsilon) \nabla_\phi f(g(\epsilon, \phi, x)) d\epsilon \\
&= E_{\epsilon \sim p(\epsilon)}[\nabla_\phi f(z)].
\end{aligned}$$

Finalmente, tenemos un estimador simple de Monte Carlo

$$\nabla_\phi E_{z \sim q_\phi(z|x)}[f(z)] \simeq \nabla_\phi f(z).$$

De esta manera, reparametrizamos la aleatoriedad de z y ahora podremos hacer *backpropagation* por z y calcular los gradientes $\nabla_\phi f(z)$.

Nota 05. En este capítulo se enseña el *reparametrization trick* para una f cualquiera. En la sección que viene se usa una f del problema. ■

Gradiente de ELBO

Con el *reparametrization trick*, podemos escribir

$$\begin{aligned}
\mathcal{L}_{\theta, \phi}(x) &= E_{z \sim q_\phi(z|x)}[\log p_\theta(x, z) - \log q_\phi(z|x)] \\
&= E_{\epsilon \sim p(\epsilon)}[\log p_\theta(x, z) - \log q_\phi(z|x)].
\end{aligned}$$

Luego, podemos obtener un estimador simple de Monte Carlo, $\tilde{\mathcal{L}}_{\theta, \phi}(x)$, de $\mathcal{L}_{\theta, \phi}(x)$ para cada $x \in \mathcal{D}$ (o $\mathcal{M}$):

$$\begin{aligned}
\epsilon &\sim p(\epsilon) \\
z &= g(\phi, x, \epsilon) \\
\tilde{\mathcal{L}}_{\theta, \phi}(x) &= \log p_\theta(x, z) - \log q_\phi(z|x).
\end{aligned}$$

Finalmente, para optimizar el ELBO se ocupan los gradientes $\nabla_{\theta, \phi} \tilde{\mathcal{L}}_{\theta, \phi}(x)$.

Algoritmo Auto-Encoding Variational Bayes (AEVB)

Por último, el algoritmo estocástico para optimizar el ELBO es

Algoritmo 06. (AEVB).

Data:

- $\mathcal{D}$: Dataset.
- $q_\phi(z|x)$: Modelo de inferencia.
- $p_\theta(x, z)$: Modelo generativo.

Resultado:

- θ, ϕ : Parámetros aprendidos
1. $(\theta, \phi) \leftarrow$ Inicializar parámetros.

2. **while** SGD *no ha convergido* **do**:
3. $\mathcal{M} \sim \mathcal{D}$ (extraer un minibatch random).
4. $\epsilon \sim p(\epsilon)$ (extraer sonido random para cada punto en $\mathcal{M}$).
5. Calcular $\tilde{\mathcal{L}}_{\theta, \phi}(\mathcal{M}, \epsilon)$ y sus gradientes $\nabla_{\theta, \phi} \tilde{\mathcal{L}}_{\theta, \phi}(\mathcal{M}, \epsilon)$.
6. Actualizar θ, ϕ usando un optimizador SGD.

Glosario

- **Inferencia Variacional:**
- **Estimador de Monte-Carlo:**

Neural Discrete Representation

El siguiente contenido está basado en el paper “*Neural Discrete Representation*” de Aaron van den Oord, Oriol Vinyals y Koray Kavukcuoglu (2018).